

→ AIR-TRAFFIC

Air-Traffic: Enterprise AI Control Plane & Observability Layer

Research Analysis, Architecture Design & Product Specification
— June 2026

Research Analysis · Architecture · Product Specification

Contents

Air-Traffic: Enterprise AI Control Plane & Observability Layer

Research Analysis, Architecture Design & Product Specification

1. Executive Summary
2. The Three Control Planes
3. Vendor Analysis
4. Cross-Vendor Synthesis
5. Air-Traffic API: VendorAdapter Architecture
6. Baseline Configuration Profiles
7. Observability Layer Design
8. UI Layout Proposals
9. Killer Differentiating Features
10. Appendix: Stub Disposition Reference

Air-Traffic: Enterprise AI Control Plane & Observability Layer

Research Analysis, Architecture Design & Product Specification

Document status: Research-verified — all vendor capabilities confirmed against live documentation (June 2026).

Application: Air-Traffic — unified control plane and observability layer for enterprise AI.

Architecture note: Air-Traffic’s API is modeled on the `it-scorecard` connector/emitter pattern: each vendor/platform is a `VendorAdapter` (a typed connector) that emits `ops-observation-batch/v1` signal in synthetic or proxy mode. Controls the vendor does not natively expose are fulfilled by **pushing managed configuration into the dev/agent environment** (`EnvManaged`) — not by intercepting traffic. An inline gateway (`ProxyEnforced`) exists only as an optional module for two runtime-only controls. Every capability carries its disposition, so coverage is always explicit and auditable.

1. Executive Summary

Enterprise AI adoption has fractured into a sprawling landscape: model APIs from a dozen vendors, agentic platforms with divergent primitives, and no consistent governance surface. An admin who wants to enforce a no-training-data policy across OpenAI, Anthropic, and Bedrock must visit three consoles, apply three different workflows, and hope the settings hold — with no unified audit trail to prove it.

Air-Traffic is an AI control plane and observability application that solves this. It exposes a single API (the `VendorAdapter` interface, modeled on the `it-scorecard` connector/emitter pattern) through which an enterprise can:

- **Control** model access, data policy, content safety, agentic primitives (MCP servers, hooks, cron jobs, code reviewers), and spend limits — across every major AI vendor from one surface.
- **Observe** usage metering, cost, audit events, and agentic action traces — in a unified, normalized `ops-observation-batch/v1` stream.
- **Recommend** industry-appropriate rigor baselines (fintech, healthcare, general SaaS) as one-click starting configurations.

Air-Traffic fulfils every control through whichever of **four mechanisms** fits, and always says which:

DISPOSITION	MECHANISM	EXAMPLES
<code>VendorNative</code>	drive the vendor’s native admin API	<code>model_permissions</code> , Bedrock Guardrails, <code>spend_alerts</code> , content filters, audit export
<code>EnvManaged</code>	push managed config into the developer/agent environment and	

DISPOSITION	MECHANISM	EXAMPLES
	read it back (no request interception)	MCP allow/deny, shared hooks, code-review gate, cron, per-environment model assignment
ProxyEnforced	intercept the request — optional inline gateway, off by default	hard per-user cross-vendor spend cap, per-request PII scrub
MonitorOnly	verify, gate, and alert — Air-Traffic cannot set it	contractual training opt-out, ZDR-via-DPA, vendor-side retention/residency, soft spend alerts, read-only audit gaps

Architecture scope (important): Air-Traffic's spine is control-plane (drive vendor admin APIs) + config-distributor (push agentic config into environments) + observability — it is not an inline inference proxy on the critical path. The ProxyEnforced gateway is a clearly-bounded optional module for the two residual runtime-only controls; nothing in the spine depends on it. See `air-traffic-system-design.md §11`.

Research finding: The majority of enterprise AI governance controls — particularly the developer-workflow plane (agentic primitives) — are not exposed by vendor admin APIs. But the right answer is rarely a proxy: agentic primitives are governed EnvManaged (managed-config distribution), and most data-policy controls are VendorNative or monitored. Only two controls genuinely need request interception — which is why the inline gateway is optional, not foundational.

2. The Three Control Planes

Air-Traffic organizes every control and observable into three planes. Each plane maps to a distinct governance concern and buyer persona.

2.1 Developer-Workflow Plane

Owner: VP Engineering / Platform Engineering
Concern: What can developers and agents do? Which models, which tools, under what review requirements?

CONTROL	DESCRIPTION
Model access assignment	Which models users, teams, and projects may call
User/team tier	Standard / elevated / admin tiers with different model availability
Test requirements	Mandatory test coverage before agent commits are accepted
Code-review level	Human-required / agentic-reviewer / none
MCP server policy	Allow/deny lists and scope constraints for MCP servers

CONTROL	DESCRIPTION
Shared hooks	Org-level hooks pushed into every dev/agent environment via managed config
Cron/scheduled jobs	Org-managed scheduled agent jobs
Agentic code reviewers	Configured reviewers for agentic code review workflows

Research finding: Few vendors expose org-level MCP policy, shared hooks, or agentic code-review controls via a programmatic admin API. But these do **not** require a proxy — they are governed by `EnvManaged` : Air-Traffic renders managed configuration and pushes it into the developer/agent environment (e.g. Claude Code managed settings for MCP allow/deny + hooks; GitHub branch protection for a required-review gate; CI for cron and test-coverage gates), then reads the effective state back for drift. Native exceptions exist (Databricks Unity AI Gateway; Mistral per-workspace MCP Connectors; GitHub Copilot MCP policy), but the general mechanism is config distribution, not request interception.

2.2 Data-Policy Plane

Owner: CISO / Data Privacy Officer
Concern: Where does our data go? What is logged? Who trains on it?

CONTROL	DESCRIPTION
Training opt-out	Prevent org’s requests from being used to train models
Data retention	How long request/response data is held by the vendor
Request/response logging	Whether and how the vendor logs prompts/completions
Data residency	Which regions data is processed and stored in
PII/PHI handling	Detection and redaction of sensitive data
Content safety policy	Org-level safety filter configuration
Zero-data-retention (ZDR)	API calls never logged or retained by the vendor

Rigor levels:

LOCK	LEVEL	MEANING
	Standard	Training opt-out on; 30-day retention; basic content filters
	Elevated	ZDR where available; 7-day retention; PII redaction at proxy; stricter filters
	Strict	ZDR (VendorNative where offered, else <code>MonitorOnly</code> + gate); zero

LOCK	LEVEL	MEANING
		retention; full PII/PHI scrub (ProxyEnforced); maximum content filters; all requests audited

2.3 Budget Plane

Owner: FinOps / Engineering Leadership
Concern: How much are we spending, per team and per vendor?

CONTROL	DESCRIPTION
Org-level spend cap	Hard monthly limit across all vendor calls
Team/project spend cap	Per-team or per-project monthly sub-limit
Per-user spend cap	Individual developer spend limit
Token rate limit	Tokens per minute per user/team
Soft vs. hard cap	Soft = alert + throttle; hard = block
Alerting thresholds	Percent-of-budget triggers for notifications
Cost attribution	Tag every request with team/project/user for chargeback

Research finding: Only OpenAI and Azure expose programmatic spend alert APIs. AWS Bedrock has no native hard spend cap at all — the most critical budget gap in the market. Per-user and per-team sub-limits are **monitored** by Air-Traffic from vendor usage APIs (with soft alerts) by default; a true hard mid-request stop is the one budget control that needs the optional gateway (**ProxyEnforced**).

3. Vendor Analysis

Dispositions:

VendorNative = vendor exposes a programmatic admin API (Air-Traffic drives it).

EnvManaged = no vendor API, but the control lives in the dev/agent environment — Air-Traffic pushes managed config and reads it back (MCP, hooks, code-review, cron). No request interception.

ProxyEnforced = needs the **optional** inline gateway (runtime-only: hard cross-vendor caps, per-request PII scrub).

MonitorOnly = Air-Traffic cannot set it (no admin API, not env-config, no proxy used), but **verifies, gates, and alerts** — e.g. contractual training opt-out, ZDR-via-DPA, vendor-side retention/residency, soft spend alerts, and read-only audit/ log gaps.

Unverified = could not confirm from current live docs.

3.1 OpenAI

Enterprise surface: OpenAI Platform — Organizations + Projects + Admin API

Enterprise API Docs: <https://developers.openai.com/api/docs/guides/admin-apis>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
User invite/list/update/remove	VendorNative	<code>POST /admin/organization/invites ; GET/PATCH/DELETE /admin/organization/users/{id}</code>
Project create/update/archive	VendorNative	<code>POST /admin/organization/projects ; PATCH /admin/organization/projects/{id}</code>
Model access per project (allowlist/denylst)	VendorNative	<code>PATCH /admin/organization/projects/{id}/model_permissions</code> with <code>mode: allow_list\ deny_list</code> and model list
Admin API key management	VendorNative	<code>/admin/organization/admin_api_keys</code>
Project API key management	VendorNative	<code>/admin/organization/projects/{id}/api_keys</code>
Service account management	VendorNative	Referenced in admin docs
MCP server allow/deny	EnvManaged	No vendor API; pushed via the agent environment's managed config (e.g. Codex/Claude Code managed settings), read back for drift
Shared hooks / cron / code-review controls	EnvManaged	Pushed into the environment (managed settings / CI / branch protection); not a vendor API

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default ON)	API data never used for training by default — contractual; no toggle needed
Zero-data-retention (ZDR)	MonitorOnly	Enterprise-negotiated only; not self-serve; effectively applies <code>store=false</code> on completions
Data residency (10 regions)	VendorNative (dashboard, not <code>admin API</code>)	EU, UK, US, CA, JP, KR, SG, IN, AU, UAE; set by creating a new Project + selecting region (existing projects immutable); EU residency is in-region with ZDR ; ~10% uplift

CONTROL	STUB	NOTES
		for residency-eligible models released $\geq$ Mar 5 2026
Abuse monitoring opt-out	MonitorOnly	Dashboard setting only (<code>Settings</code> $\rightarrow$ <code>Org</code> $\rightarrow$ <code>Data controls</code>); no admin API endpoint confirmed
PII/PHI handling	ProxyEnforced	No per-request PII masking via admin API; BAA available
Content safety policy (org-level)	ProxyEnforced	No org-level safety policy API; filtering at model level only

BUDGET CONTROLS

CONTROL	STUB	NOTES
Spend alerts per project	VendorNative	<code>POST /admin/organization/projects/{id}/spend_alerts</code> ; thresholds in cents; email notifications; monthly/custom intervals
Rate limits per project (read + set)	VendorNative	<code>GET/POST /admin/organization/projects/{id}/rate_limits</code> ; set per-model $\leq$ org limits
Hard spend caps	ProxyEnforced	Confirmed negative (2026): the org monthly budget no longer hard-stops — it alerts/emails and keeps serving + billing. The only native hard stop is prepaid credits with auto-recharge OFF (org-wide, lagging). Real per-key/project hard caps require the Air-Traffic gateway
Usage/cost metering API	VendorNative	<code>GET /v1/usage/*</code> (completions, audio, embeddings, images); breakdown by project/model/API key; also <code>/v1/organization/costs</code>

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Audit logs	VendorNative	<code>GET /admin/organization/audit_logs</code> ; 51 event types (api_key., login., project., user.); cursor-paginated; Enterprise tier
Usage metering	VendorNative	<code>/v1/organization/usage</code> — per model, project, user, API key; daily granularity

SIGNAL	AVAILABILITY	NOTES
Cost metering	VendorNative	<code>/v1/organization/costs</code> — per project, per model
Request/response logs (API platform)	MonitorOnly	Logs stored briefly for abuse monitoring; not customer-retrievable via API. Note: ChatGPT Enterprise (separate product, §3.12) exposes full conversation/file export via the Compliance Logs Platform
Agentic action logs (MCP, hooks)	MonitorOnly	No native log stream for agent actions on the API platform; ChatGPT Enterprise Compliance Logs now include Codex/agent usage

2026 additions (API platform): the **Global Admin Console** (`admin.openai.com`) adds tenant-level control spanning multiple ChatGPT Enterprise workspaces (custom roles, directory-synced groups, app/action enable-disable). The **Compliance Logs Platform** exports immutable, time-windowed JSONL (Admin Audit, User Authentication, Codex Usage). Both are primarily ChatGPT Enterprise surfaces — see §3.12.

3.2 Anthropic

Enterprise surface: Claude API (`platform.claude.com`) with Admin API, Usage/Cost API, Compliance API (Enterprise); `claude.ai` Enterprise with Analytics/Compliance Access API

Enterprise API Docs: - Admin API: <https://platform.claude.com/docs/en/manage-claude/admin-api> - Usage & Cost API: <https://platform.claude.com/docs/en/manage-claude/usage-cost-api> - Compliance API: <https://platform.claude.com/docs/en/manage-claude/compliance-api> - Rate Limits API: <https://platform.claude.com/docs/en/manage-claude/rate-limits-api> - Data Residency: <https://platform.claude.com/docs/en/manage-claude/data-residency>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
User invite/list/update/remove	VendorNative	<code>POST /v1/organizations/invites</code> ; <code>GET/PATCH/DELETE /v1/organizations/users/{id}</code> ; 5 org roles
Workspace create/list/archive	VendorNative	<code>POST /v1/organizations/workspaces</code> ; max 100 per org; archive is irreversible
Workspace member management	VendorNative	<code>POST/DELETE /v1/organizations/workspaces/{id}/members</code>
API key list/update (not create)	VendorNative	

CONTROL	STUB	NOTES
		<code>GET/POST /v1/organizations/api_keys</code> ; keys created in Console only
Workload Identity Federation (OIDC)	VendorNative	<code>org:admin</code> OAuth; OIDC issuer + federation rule CRUD
Model access controls per workspace	EnvManaged / ProxyEnforced	No model permission API at workspace level (biggest gap vs. OpenAI). For agentic use, restrict models via Claude Code managed settings (EnvManaged); for raw API access, the optional gateway (ProxyEnforced)
Rate limits (read-only)	VendorNative	<code>GET /v1/organizations/workspaces/{id}/rate_limits</code> ; read-only — changes require Console
MCP tunnel management	VendorNative (limited)	OAuth <code>org:manage_tunnels</code> ; max 10 active tunnels/org
Shared hooks / cron / code-review controls	EnvManaged	Claude Code managed settings (hooks) + GitHub branch protection / CI (review gate, cron); not an Anthropic API

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default ON)	API traffic never used for training — contractual default
Inference geo per-request	VendorNative	<code>inference_geo: "us" "global"</code> on <code>/v1/messages</code> ; priced at 1.1x for "us" across all token categories; supported only on Claude Opus 4.6 / Sonnet 4.6 and later (older models return 400)
Workspace inference geo controls	VendorNative	<code>allowed_inference_geos</code> and <code>default_inference_geo</code> per workspace
Workspace geo (storage at rest)	VendorNative (limited)	Set at workspace creation; US only currently; immutable after creation
ZDR	MonitorOnly	Available for eligible orgs; not self-serve API toggle
API log retention (7-day default)	MonitorOnly	Configurable to 30 days via DPA only; not a self-serve programmatic toggle

CONTROL	STUB	NOTES
Content safety policy (org-level)	ProxyEnforced	No admin API for safety config; system prompt is the only tuning surface
PII/PHI handling	ProxyEnforced	No per-request PII detection via admin API; BAA available

BUDGET CONTROLS

CONTROL	STUB	NOTES
Usage metering	VendorNative	<code>GET /v1/organizations/usage_report/messages</code> ; filter by API key, workspace, model, service tier, inference geo; 1m/1h/1d buckets; up to 31-day window; 5-min data freshness
Cost reporting	VendorNative	<code>GET /v1/organizations/cost_report</code> ; USD breakdown by workspace; daily only
Claude Code analytics (per-user)	VendorNative	Claude Code Analytics API; per-user commits, PRs, lines of code, session counts
Rate limits (read-only)	VendorNative	<code>GET /v1/organizations/rate_limits</code> ; groups by model tier, batch, skills, web search
Workspace spend limits	MonitorOnly	Confirmed Console-only: workspace spend/limit overrides are set in Console → Limits tab; no programmatic SET endpoint
Hard token/spend caps	ProxyEnforced	No hard-cutoff enforcement API
Spend alerting	Unverified	No confirmed programmatic spend alert creation API

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage metering	VendorNative	<code>/v1/organizations/usage_report/messages</code> — rich filtering
Cost metering	VendorNative	<code>/v1/organizations/cost_report</code> — USD, daily
Compliance Activity Feed	VendorNative (Enterprise)	<code>GET /v1/compliance/activities</code> ; 30+ event types; identity, org config, project lifecycle, conversation lifecycle; 180-day retention

SIGNAL	AVAILABILITY	NOTES
Chat/file/project content retrieval	VendorNative (Enterprise)	<code>/v1/compliance/chats</code> , <code>/v1/compliance/files</code> , <code>/v1/compliance/projects</code> ; read + delete
Directory (users, roles, groups)	VendorNative (Enterprise)	<code>/v1/compliance/organizations</code> , <code>/v1/compliance/users</code> , <code>/v1/compliance/roles</code>
Enterprise Analytics API (Mar 2026)	VendorNative (Enterprise)	Per-user cost/usage/engagement spanning chat, Claude Code , Cowork , and Office agents ; up to 90 days
Request/response logs (API)	MonitorOnly	7-day retention; not customer-retrievable via API
Agentic action logs (MCP, hooks)	MonitorOnly	Not exposed via any confirmed API endpoint

Key insight: Anthropic's Compliance API (Enterprise only) is unusually deep — it returns actual chat content, files, and project data, not just metadata. This distinguishes Anthropic strongly for legal/compliance workflows. But the developer-workflow plane is almost entirely **EnvManaged** (governed via managed config in the agent environment, e.g. Claude Code managed settings).

3.3 Google Gemini / Vertex AI

Enterprise surface: Google Cloud IAM + Organization Policies; Gemini Enterprise Agent Platform (evolved Vertex AI); Cloud Audit Logs; Cloud Billing API; VPC Service Controls

Enterprise API Docs: - Vertex AI: <https://cloud.google.com/vertex-ai/docs> - IAM / access: <https://cloud.google.com/vertex-ai/generative-ai/docs/access-control> - VPC Service Controls: <https://docs.cloud.google.com/vertex-ai/docs/general/vpc-service-controls> - Model Armor (content-safety enforcement, GA 2026): <https://docs.cloud.google.com/model-armor/release-notes> - Data residency: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/data-residency> - Zero data retention: <https://docs.cloud.google.com/gemini-enterprise-agent-platform/resources/zero-data-retention> - Spend Caps blog (Next '26): <https://cloud.google.com/blog/topics/cost-management/introducing-spend-caps-ai-cost-visibility-next26> - Tool governance: <https://cloud.google.com/blog/products/ai-machine-learning/new-enhanced-tool-governance-in-vertex-ai-agent-builder>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Model access (IAM roles)	VendorNative	Roles: <code>aiplatform.admin</code> , <code>aiplatform.user</code> , <code>aiplatform.endpoints.predict</code> ; Consumer Procurement Entitlement Manager role to enable open models in Model Garden

CONTROL	STUB	NOTES
Per-team model restriction	VendorNative	IAM at org/folder/project level; ABAC via tags; custom roles via IAM
Service account API key scoping	VendorNative	Service accounts with least-privilege IAM; Gemini Role Picker (2026) generates least-privilege roles from natural language
Vertex AI Agent Engine (agentic controls)	VendorNative	IAM-controlled deployment; VPC-SC compliant; authentication config
Tool governance (API registry)	VendorNative	Cloud API Registry integration in Agent Builder Console; admin curates approved tools; org-level allow/deny for agent tools
MCP server allow/deny	EnvManaged	Tool governance is API-registry-based, not MCP-protocol-native; MCP allow/deny pushed via the agent environment's managed config
Shared hooks / cron / code-review controls	EnvManaged	Pushed into the environment (managed settings / CI / branch protection)

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (contractual)	"Will not use your data to train/fine-tune models without prior permission" — DPA; no API toggle
ZDR (per-request)	VendorNative	<code>store=false</code> parameter per request — self-serve
ZDR (org-level)	VendorNative (form-gated)	Full org-level ZDR: form request or Invoiced Billing; not yet self-serve via API
Data Access audit logs	VendorNative	Disabled by default — admins must enable "Data Read" audit logs; may incur cost. Common misconfiguration.
Data residency	VendorNative	Data Residency Zone (DRZ); CMEK supported (not preview models)
VPC Service Controls	VendorNative	Full VPC-SC support; perimeter blocks public internet; allowlist via access policies
PII/PHI handling	VendorNative + ProxyEnforced	Cloud DLP API available; not auto-applied to Vertex inference —

CONTROL	STUB	NOTES
		inline redaction needs the optional gateway
Per-request safety filters	ProxyEnforced (org-level)	Per-caller <code>safetySettings</code> override possible — the in-model filters can't be locked org-wide
Content safety / prompt-injection (org-enforced)	VendorNative (Model Armor, GA 2026)	Model Armor (GA 2026) screens prompts/responses for safety + prompt-injection/jailbreak and is org-enforceable via Vertex integration + a monitoring dashboard — closes the per-caller-override gap

Gap (narrowed in 2026): The in-model `safetySettings` can still be overridden per-caller, but **Model Armor** (GA 2026) now provides an org-enforceable safety + prompt-injection screen at the Vertex layer. Where Model Armor is not adopted, org-wide enforcement would need the optional Air-Traffic gateway.

BUDGET CONTROLS

CONTROL	STUB	NOTES
Quota / rate limits	VendorNative	Cloud Service Usage API; <code>serviceusage.quotas.update</code> permission; quota increases via console or API
Hard spend caps (project-level)	VendorNative (private preview)	Announced at Next '26 (April 2026); project-level hard caps; pauses API traffic when budget reached; resources intact
Soft spend alerts	VendorNative	Cloud Billing Budgets API; Pub/Sub or email notification
Per-user/team token caps	ProxyEnforced	Spend Caps are project-level only; per-user enforcement requires proxy
Cost metering	VendorNative	Cloud Billing API + BigQuery billing export; per-project, per-label, per-service

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage metering	VendorNative	Cloud Monitoring AI metrics (token counts, latency, error rates per model/project); Cloud Billing API
Cost metering	VendorNative	Cloud Billing API; BigQuery export

SIGNAL	AVAILABILITY	NOTES
Audit logs	VendorNative	Cloud Audit Logs (Admin Activity always on; Data Access must be enabled and paid)
Request/response logs	VendorNative (off by default)	Cloud Logging when Data Access logs enabled
Agentic action logs	VendorNative	Vertex AI Agent Engine logs to Cloud Logging; tool call traces via Cloud Trace
Metrics/monitoring exports	VendorNative	Cloud Monitoring; export to BigQuery, Pub/Sub, Elastic integration confirmed
Alerting	VendorNative	Cloud Monitoring alerting policies; Spend Cap alerts

3.4 AWS Bedrock

Enterprise surface: AWS IAM + Service Control Policies (SCPs); Bedrock Guardrails; AgentCore Policy; CloudTrail; CloudWatch; AWS Budgets

Enterprise API Docs: - Bedrock: <https://docs.aws.amazon.com/bedrock/latest/userguide/> - IAM / security: <https://docs.aws.amazon.com/bedrock/latest/userguide/security-iam.html> - Model invocation logging: <https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html> - Guardrails: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html> - Cross-account guardrails: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-enforcements.html> - AgentCore Policy + Guardrails (GA June 2026): <https://aws.amazon.com/about-aws/whats-new/2026/06/amazon-bedrock-agentcore-policy-guardrails-generally-available/>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Model access via IAM / SCPs	VendorNative	IAM <code>bedrock:InvokeModel</code> conditions on model ARNs; SCPs at org level restrict across all accounts simultaneously. Model access page retired 2026 — models available by default in a region; governance is purely IAM/SCP
ABAC (tag-based policies)	VendorNative	Full ABAC support
Bedrock Agents (tool use)	VendorNative	Agents: IAM controls invocation; action groups explicitly configured per agent
AgentCore Policy	VendorNative	Tool-use policies authored via natural language or policy-as-code; default-deny — tool access must be explicitly granted; enforced at the gateway perimeter, outside

CONTROL	STUB	NOTES
		the model's reasoning loop (so it's immune to prompt injection); intercepts every tool call. (Unverified: the "compiled to Cedar" detail and a specific GA date are not confirmed by the cited release page.)
AgentCore Guardrails in policy	VendorNative	"Amazon Bedrock AgentCore now supports Bedrock Guardrails in policy" (release dated June 17, 2026); evaluates every authorized agent action output and gateway call input at the AgentCore perimeter
MCP server allow/deny (native)	Unverified	AgentCore Policy covers tool-level restrictions; native MCP allow/deny at org level not confirmed
Shared hooks / cron controls	EnvManaged	Pushed into the environment (managed settings / CI); not a Bedrock API

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default)	Customer data never used for training by default; fine-tuned models get private copy per customer
ZDR (implicit)	VendorNative	Bedrock does not retain inference data by default; invocation logging is opt-in
Request/response logging	VendorNative	<code>PutModelInvocationLoggingConfiguration</code> API; disabled by default; destination: S3 + CloudWatch Logs; captures full request/response (up to 100KB inline), token counts, <code>identity.arn</code>
Data residency	VendorNative	Region-scoped; VPC endpoints via PrivateLink; CMEK supported; SCPs enforce region constraints
PII/PHI handling	VendorNative	Bedrock Guardrails sensitive info filters: REDACT / ANONYMIZE / BLOCK PII entities and custom regex patterns
Content safety policy (deployment-level)	VendorNative	Bedrock Guardrails: content filters (hate, violence, sexual, misconduct, prompt attack) per severity threshold
Cross-account/org Guardrails	VendorNative (GA April 2026)	Central admin creates guardrail → attaches to <code>BEDROCK_POLICY</code> via AWS Organizations → auto-applies to all model invocations across all OUs/accounts; layered (most restrictive wins)

BUDGET CONTROLS

CONTROL	STUB	NOTES
Hard spend caps	ProxyEnforced	No native hard spend cap in Bedrock — the most significant enterprise gap. AWS's own prescribed pattern is an API Gateway/Lambda proxy checking budget in DynamoDB (a “proactive AI cost management” reference architecture) — i.e. literally the Air-Traffic gateway model
Soft spend alerts	VendorNative	AWS Budgets filtered by tags: SNS/email/chat at threshold; alerting only, not enforcement
Token/request quotas	VendorNative	Service Quotas API: tokens/min, requests/min, concurrent invocations; increase via quota requests
Per-user/team cost attribution	VendorNative	<code>identity.arn</code> in invocation logs; Application Inference Profiles for per-team/app cost tagging; cost-by-IAM-principal in CUR 2.0 / Cost Explorer (2026); request metadata tagging
Cost metering	VendorNative	Cost Explorer API; CUR; CloudWatch Bedrock metrics (InputTokenCount, OutputTokenCount, InvocationLatency)

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage metering	VendorNative	CloudWatch <code>aws/bedrock</code> namespace; per model metrics
Cost metering	VendorNative	Cost Explorer API; CUR; per-model per-account
Audit logs	VendorNative	CloudTrail: all admin + data events (InvokeModel, CreateGuardrail, etc.)
Request/response logs	VendorNative	Invocation logging API; full prompt + completion; queryable via CloudWatch Logs Insights or Athena
Agentic action logs	VendorNative	Bedrock Agents trace events; AgentCore Policy logs tool access decisions; guardrail interventions

SIGNAL	AVAILABILITY	NOTES
Guardrail monitoring	VendorNative	CloudWatch metrics for guardrail interventions; CloudTrail for <code>ApplyGuardrail</code>
Monitoring/alerting	VendorNative	CloudWatch alarms; AWS Budgets

Standout capabilities: (1) Cross-account Guardrails via Organizations `BEDROCK_POLICY` is the most powerful org-level safety control in any AI vendor's stack — immutable versioned guardrails, layered union with most-restrictive-wins, `included_models` / `excluded_models` scoping, `DescribeEffectivePolicy` to verify. (2) AgentCore tool governance is architecturally strong: policy enforced at the gateway perimeter, outside the model's reasoning loop — immune to prompt injection. (3) But no native hard spend cap remains the critical gap — and AWS's own guidance is to build the proxy Air-Traffic provides.

3.5 Azure OpenAI Service

Enterprise surface: Azure Resource Manager + RBAC; Azure Monitor; Content Filters API; Diagnostic Settings; Azure Budgets

Enterprise API Docs: - Azure OpenAI: <https://learn.microsoft.com/en-us/azure/ai-services/openai/> - RBAC: <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/role-based-access-control> - Quota/TPM: <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/quota> - Content Filters: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/content-filter-configurability> - Monitoring: <https://learn.microsoft.com/en-us/azure/foundry/openai/monitor-openai-reference>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Model access (RBAC)	VendorNative	4 built-in roles: <code>Cognitive Services OpenAI User</code> , <code>OpenAI Contributor</code> , <code>Cognitive Services Contributor</code> , <code>Usages Reader</code> ; assignable at resource/resource group/subscription/management group via ARM API
Model deployment create/edit	VendorNative	<code>Cognitive Services Contributor</code> or <code>OpenAI Contributor</code> required; REST API for deployment management
Per-user model routing	ProxyEnforced	RBAC controls WHO can manage deployments, not which model an end user calls at request time; proxy required
Provisioned Throughput Units (PTU)	VendorNative	Managed via ARM API; separate <code>ProvisionedManaged</code> SKU

CONTROL	STUB	NOTES
Content filter guardrail management	VendorNative (restricted)	Cognitive Services Contributor can create customized guardrails; full filter disable requires Microsoft Limited Access Review approval
MCP server allow/deny	EnvManaged	No Azure API; pushed via the agent environment's managed config
Shared hooks / cron / code-review controls	EnvManaged	Pushed into the environment (managed settings / CI / branch protection)

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default)	Azure OpenAI does NOT use customer data for training; Azure DPA/OST
Request/response logging	VendorNative (off by default)	Diagnostic Settings RequestResponseLog category → Log Analytics / Storage Account / Event Hub; must be explicitly enabled per resource — common misconfiguration
Data residency	VendorNative	Deploy to specific Azure region; Azure Policy enforces region restrictions
Abuse monitoring opt-out	VendorNative (gated)	Modified abuse monitoring available for eligible customers; requires request via Azure portal
Content safety filters	VendorNative	Configurable per deployment (hate, violence, sexual, self-harm — threshold per category: low/medium/high/block); full disable requires Limited Access approval
PII/PHI handling	ProxyEnforced	No native PII redaction in Azure OpenAI itself; separate Azure AI Content Safety service or proxy required

BUDGET CONTROLS

CONTROL	STUB	NOTES
TPM quota management (read)	VendorNative	Usages API: <code>GET /subscriptions/{sub}/providers/Microsoft.CognitiveServices/locations/{loc}/usages?api-version=2024-10-01</code> ; returns

CONTROL	STUB	NOTES
		<code>currentValue</code> / <code>limit</code> per model/region
Model Capacity lookup	VendorNative	Model Capacities API: <code>GET .../modelCapacities?api-version=2024-10-01&modelFormat=...</code>
Deployment TPM assignment	VendorNative	ARM API for deployment creation/update includes TPM allocation
Org/subscription spend caps	VendorNative (indirect)	Azure Cost Management Budgets with action groups (email, webhook, Logic App); no native OpenAI-level spend cap — requires automation to disable deployment
Budget alerting	VendorNative	Azure Monitor alerts; Cost Management alert rules via REST API
Per-user rate limit enforcement	ProxyEnforced	Deployment-level TPM enforced aggregate; per-user sub-limits require proxy

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage metering (tokens)	VendorNative	Azure Monitor metrics: <code>TokensProcessed</code> , <code>GeneratedTokens</code> , <code>ProcessedPromptTokens</code> ; <code>x-ratelimit-*</code> headers on every response
Cost metering	VendorNative	Azure Cost Management API; per resource, per tag
Audit logs	VendorNative	Azure Monitor Activity Log: all control-plane operations; queryable via Log Analytics REST API
Request/response logs	VendorNative	<code>RequestResponseLog</code> diagnostic category; captures <code>requestPayload_s</code> and <code>responsePayload_s</code> ; must be explicitly enabled
Agentic-action logs	Unverified	Azure AI Foundry agent tracing exists; enterprise-level agentic audit log API not confirmed
Rate limit telemetry	VendorNative	<code>x-ratelimit-limit-tokens</code> , <code>x-ratelimit-remaining-tokens</code> , <code>x-ratelimit-remaining-requests</code> in every response header

3.6 GitHub Copilot

Enterprise surface: GitHub Enterprise / Business admin center; Copilot REST API; Audit Log API; Metrics API

Enterprise API Docs: - Policies: <https://docs.github.com/en/copilot/concepts/policies> - Content Exclusions API: <https://docs.github.com/en/enterprise-cloud@latest/rest/copilot/copilot-content-exclusion-management> - Metrics API: <https://docs.github.com/en/rest/copilot/copilot-usage-metrics> - User Management API: <https://docs.github.com/en/rest/copilot/copilot-user-management>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Seat management (assign/revoke)	VendorNative	<code>GET /enterprises/{enterprise}/copilot/billing/seats</code> ; <code>POST/DELETE /orgs/{org}/copilot/billing/selected_users</code>
Feature policy controls (Chat, CLI, Cloud Agent, Extensions)	VendorNative (portal only)	Enterprise AI Controls tab; no documented REST API for policy toggles — portal-only confirmed
Model selection policy	VendorNative (portal)	Enterprise configures available models via AI Controls tab; no confirmed REST API
MCP server policy	VendorNative (portal)	MCP section in AI Controls tab (GA); controls MCP in Copilot; does NOT govern GitHub MCP server in third-party host apps
Content exclusions (file/repo paths)	VendorNative	Full REST API: <code>GET/PUT /enterprises/{enterprise}/copilot/content_exclusion</code> ; also <code>GET/PUT /orgs/{org}/copilot/content_exclusion</code> ; repo + file path glob patterns
Agentic code-review level / test gate	EnvManaged	GitHub branch protection (required reviewers) + required status checks (test-coverage gate) — config, not a Copilot setting

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default)	GitHub Copilot Business/Enterprise does NOT use prompts/suggestions for training; contractual
Telemetry opt-out	VendorNative	Org settings can disable sending telemetry
Public code filter (duplicate detection)	VendorNative	Admin API policy: block suggestions matching public code

CONTROL	STUB	NOTES
Data residency	Unverified	GitHub.com data residency options exist; enterprise Copilot-specific residency controls not clearly documented
Request/response logging (content)	MonitorOnly	GitHub does not expose prompt/response content to enterprise admins; audit log captures metadata only

BUDGET CONTROLS

CONTROL	STUB	NOTES
AI credit usage billing (2026)	VendorNative	Usage-based billing live June 1 2026; 1 AI Credit = \$0.01; Enterprise = \$39/user/mo incl. \$39 in AI Credits; usage metered by token consumption (input/output/ cached) at per-model API rates; legacy per-request overage \$0.04; <code>GET /enterprises/{enterprise}/billing/usage</code>
Seat billing tracking	VendorNative	Billing seats API
Usage metrics	VendorNative	<code>GET /enterprises/{enterprise}/copilot/metrics</code> (API version 2026-03-10); 28-day rolling, up to 1-year historical; by team, language, editor
Hard spend caps / rate limits	ProxyEnforced	No native hard cap on AI credit spend; no per-user/team rate limit API

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage metrics	VendorNative	Copilot Metrics API: active users, completions shown/accepted, chat turns, languages, editors; enterprise and org scope
Seat activity	VendorNative	User management API: last active date per seat
Audit logs	VendorNative	GitHub Audit Log API: Copilot seat assignments, policy changes, content exclusion changes, extension usage
Prompt/response content	MonitorOnly	Audit log contains metadata only; capturing content requires a proxy

SIGNAL	AVAILABILITY	NOTES
Agentic-action logs (agent control plane)	VendorNative (GA Feb 26 2026)	Enterprise AI Controls + agent control plane GA: audit log adds <code>actor_is_agent</code> , <code>user / user_id</code> , and <code>agent_session.task</code> (start/finish/fail) events; <code>actor:Copilot</code> filter; all sessions in last 24h visible (past the old 1,000 cap); agent commits link to full session logs (Mar 20 2026); 180-day retention

3.7 Microsoft 365 Copilot

Enterprise surface: Microsoft 365 Admin Center; Microsoft Purview (compliance/audit); Microsoft Graph API; Insider Risk Management; DSPM for AI

Enterprise API Docs: - Privacy & Data: <https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-privacy> - Data Residency: <https://learn.microsoft.com/en-us/microsoft-365/enterprise/m365-dr-service-copilot> - Purview for AI: <https://learn.microsoft.com/en-us/purview/ai-microsoft-purview> - Purview Audit: <https://learn.microsoft.com/en-us/purview/audit-copilot> - Admin Setup: <https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-setup>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
License assignment	VendorNative	Microsoft 365 Admin Center + Graph API group-based licensing: <code>POST /groups/{id}/assignLicense</code>
App-level Copilot toggle	VendorNative (portal)	M365 Admin Center: enable/disable Copilot per app (Teams, Word, Excel, Outlook); no confirmed Graph API for per-app toggles
Copilot Studio agents governance	VendorNative	Copilot Studio admin center: approve/block custom agents; security and governance docs
Plugin/extension policy	Unverified	Audit logs show <code>CreatePlugin</code> / <code>DeletePlugin</code> ; REST API for plugin policy management not confirmed
Web search enablement	VendorNative (portal)	Configurable in M365 Admin Center; API status unverified
MCP integration controls	Unverified	No org-level MCP policy API

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative (default)	M365 Copilot does NOT use customer prompts/responses for training; Microsoft Product Terms
Data retention (Copilot interactions)	VendorNative	Purview retention policies apply to prompts/responses (stored in user's Exchange Online mailbox); Purview Compliance portal or API
DLP for AI interactions	VendorNative	Purview DLP policies warn/block users from sharing sensitive info with AI apps (endpoint DLP)
Sensitivity label enforcement	VendorNative	Copilot honors sensitivity labels + encryption (requires EXTRACT usage right); automatic
Data residency	VendorNative	Advanced Data Residency (ADR) + Multi-Geo; per user's Preferred Data Location; M365 Admin Center
PII/PHI handling	VendorNative	Insider Risk Management "Risky AI usage" policy template; Communication Compliance for interaction monitoring

BUDGET CONTROLS

CONTROL	STUB	NOTES
Per-user license tracking	VendorNative	M365 Admin Center + Graph API
Usage/adoption reports	VendorNative	<code>GET /reports/microsoft365CopilotUsageUserDetail</code> (Graph Reports API); active users, feature usage by app
Token/interaction-level spend cap	MonitorOnly	M365 Copilot is license-priced; no per-interaction budgeting
Rate limits / hard usage caps	MonitorOnly	No native usage volume limits for M365 Copilot

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Unified Audit Log	VendorNative	Microsoft Purview Audit: captures Operation, AppIdentity, AppHost, AccessedResources (file IDs, URLs, sensitivity label IDs), AgentId, AgentName, AgentVersion, AISystemPlugin, Messages (prompt/response IDs, jailbreak

SIGNAL	AVAILABILITY	NOTES
		detection), ModelTransparencyDetails
Non-Microsoft AI app audit	VendorNative	AIAppInteraction recordType captures ChatGPT, Gemini, DeepSeek interactions; pay-as-you-go billing for these records; 180-day retention
Copilot Studio agent interactions	VendorNative	Distinct RecordTypes in audit log; AgentId and AgentName captured
Usage reports	VendorNative	Graph Reports API: adoption metrics (active users, feature usage)
DSPM for AI	VendorNative	Data Security Posture Management: discovers AI usage, surfaces risks, one-click policies; portal-based
Cost/billing	VendorNative	M365 billing in admin center; Graph API for subscription costs

License dependency: Orgs without E5 or Purview add-on have significantly reduced controls — E3 gives basic audit only. The full compliance story (DLP, Communication Compliance, Insider Risk, eDiscovery) requires E5/Purview Premium.

3.8 Cohere

Enterprise surface: Cohere Platform (dashboard.cohere.com); team management; spending limit in dashboard

Enterprise API Docs: <https://docs.cohere.com> | <https://cohere.com/enterprise-data-commitments>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Team roles (Owner / User)	VendorNative	Dashboard; Owner can invite/remove, manage API keys
API key management	VendorNative	Production key requires “Go to Production” form; evaluation keys self-serve
Per-user/team model access	ProxyEnforced	No native per-user model restrictions
Per-project key scoping	ProxyEnforced	No sub-org key scoping — keys are org-wide
VPC / on-prem deployment (Model Vault)	VendorNative	4 deployment modes: SaaS, Managed Cloud, VPC, On-Prem

CONTROL	STUB	NOTES
Agentic/MCP controls	Unverified	North Agent Platform exists; no admin-level MCP allow/deny documented

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out	VendorNative	Dashboard toggle: Settings → Data Controls
ZDR	MonitorOnly	Enterprise only; must email support@cohere.com to request
30-day default log retention	VendorNative	Policy-enforced; prompts/generations auto-deleted after 30 days on SaaS
PII stripping before training	VendorNative	Platform filters PII before any training use
Data residency	Unverified	“Regional data plane” mentioned in enterprise context; programmatic control not documented
Request/response log access	MonitorOnly	Cohere employees have JIT-only access; no customer-facing log API confirmed

BUDGET CONTROLS

CONTROL	STUB	NOTES
Monthly spending limit (dashboard)	VendorNative	Configurable: Billing & Usage → Spending Limit tab
Per-key spend cap	ProxyEnforced	No per-key limits; org-level only
Programmatic billing/usage API	MonitorOnly	No confirmed admin REST endpoint for spend management

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Usage dashboard	VendorNative	Dashboard-based usage reports
Audit log API	Unverified	“Audit logging” mentioned in enterprise materials; programmatic endpoint not confirmed
Request/response logs	MonitorOnly	No customer-facing log retrieval API; full capture needs the optional gateway

3.9 Mistral AI

Enterprise surface: La Plateforme (console.mistral.ai); Enterprise Admin API (Beta/Preview, Enterprise-only) — the richest programmatic admin surface of any smaller vendor

Enterprise API Docs: - Admin API: <https://docs.mistral.ai/admin/admin-api/overview> - Organizations: <https://docs.mistral.ai/admin/security-access/organization> - Audit Logs: <https://docs.mistral.ai/admin/monitor-comply/audit-logs/overview> - Billing: <https://docs.mistral.ai/admin/user-management-finops/billing>

DEVELOPER-WORKFLOW CONTROLS

CONTROL	STUB	NOTES
Organizations + Workspaces (up to 500/org)	VendorNative	Admin API: <code>/beta/admin/workspaces</code> (Preview, Enterprise-only)
User management + invitations	VendorNative	Admin API: <code>/beta/admin/users</code>
User groups for role assignment	VendorNative	Admin API: <code>/beta/admin/user-groups</code>
API key management (workspace-scoped)	VendorNative	Admin API: <code>/beta/admin/api-keys</code>
SSO / SAML	VendorNative	Configurable via Admin Panel
Connector scoping per workspace	VendorNative	Connector access and API key scoping per workspace; stops automated workloads impersonating users
Per-workspace Vibe (Le Chat) controls	VendorNative	Admin Panel: enable/disable per workspace
Per-workspace model access controls	ProxyEnforced	Confirmed negative: no model-level allow/deny per workspace; proxy required
Per-workspace Skills / Connectors / MCP Connectors	VendorNative	Rare native MCP control: admins enable/disable Skills, Connectors, and MCP Connectors per workspace via the Admin Panel — one of the only vendors with any native MCP governance

DATA-POLICY CONTROLS

CONTROL	STUB	NOTES
Training opt-out (Teams/Enterprise default)	VendorNative	Toggle in Admin Panel; Teams/Enterprise opted out by default
Chat retention policy	VendorNative	Admin Panel: configurable at 30-90 days, 180 days, or 1 year
30-day default API log retention	VendorNative	Policy: inputs/outputs kept 30 rolling days for stateless APIs

CONTROL	STUB	NOTES
ZDR	MonitorOnly	Scale plan only; request via Help Center; visible in Admin Console Privacy once approved
Data residency	VendorNative	EU-first; enterprise APIs include regional data processing controls
PII handling	Unverified	GDPR-compliant; EU provider preference; specific PII redaction API not documented

BUDGET CONTROLS

CONTROL	STUB	NOTES
Per-workspace spend limits	VendorNative	Admin Panel + Admin API billing endpoint: track consumption and set spending limits per workspace
Billing API	VendorNative	/beta/admin/billing (Preview)
Usage/analytics API	VendorNative	/beta/admin/analytics (Preview) — consumption data
Enterprise rate limits	VendorNative	Increased limits and system SLAs for enterprise

OBSERVABILITY

SIGNAL	AVAILABILITY	NOTES
Audit logs (Admin Panel)	VendorNative	Enterprise default-on; captures: auth, API key changes, org/workspace changes, user management, Vibe interactions
Audit log programmatic API	MonitorOnly	No export and no retrieval API confirmed — Admin Panel UI only; no CSV export
Usage/analytics API	VendorNative	/beta/admin/analytics covers consumption
Request/response logs	Unverified	Not documented as customer-accessible

Standout: Mistral has the most mature Admin API among smaller vendors — users, workspaces, API keys, billing, analytics all programmatic. But the audit log is panel-only with no export path — a significant gap for compliance workflows.

3.10 Meta Llama (via Hosting Providers)

Meta provides no hosted API. Controls live entirely in the hosting layer.

TOGETHER AI

Docs: <https://docs.together.ai/docs/api-keys-authentication> | <https://docs.together.ai/docs/billing-credits>

CONTROL	PLANE	STUB	NOTES
Project-scoped API keys	Developer	VendorNative	Keys scoped to projects; “key in project A can only access project A resources”
Key expiration dates	Developer	VendorNative	Settable per-key
Team roles (Admin / Member)	Developer	VendorNative	Both roles can create/ revoke keys within a project
Per-user/key spend caps	Budget	ProxyEnforced	Explicitly not available: “You can’t cap spend or rate-limit individual API keys”
Org-level usage limits	Budget	VendorNative	Apply at org level (not per-key)
Cost attribution by api_key_id	Budget	VendorNative	Inference requests return <code>api_key_id</code> for cost tracking
Training opt-out	Data-Policy	VendorNative	Training is opt-in, off by default — no training without explicit consent (Privacy & Security settings)
Zero data retention (ZDR)	Data-Policy	VendorNative	Self-serve toggle in Privacy & Security (“No” to store prompts / train)
Audit logs	Observability	MonitorOnly	Not documented

3.11 Additional Vendors with Real Enterprise Admin Surfaces

DATABRICKS — UNITY AI GATEWAY

Docs: <https://www.databricks.com/blog/ai-governance-data-ai-summit-2026-whats-new-unity-ai-gateway>

The richest native governance story for agentic AI workloads — particularly Llama/OSS models on Databricks.

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	Foundation model access policies (by provider, country, approval, tags)	VendorNative	Unity Catalog fine-grained access
Developer-Workflow	Contextual Service Policies (allow/deny/require approval for agentic actions per user/agent/model/MCP/tool/content)	VendorNative	Beta
Developer-Workflow	MCP service governance (Google Drive, Jira, Slack, GitHub, etc.)	VendorNative	Managed integrations via Unity Catalog
Data-Policy	PII exposure mitigation, prompt injection/jailbreak detection	VendorNative	Policy enforcement layer
Data-Policy	AI asset registration/discovery (models, agents, MCP, skills)	VendorNative	Unity Catalog
Budget	Hard spend caps with auto-stop	VendorNative	Announced Data + AI Summit 2026; stop requests when budget exceeded
Budget	Cost visibility by user/team/tool/use case	VendorNative	Unified cost dashboard
Observability	Unified tracing (model interactions + MCP tool activity)	VendorNative	End-to-end agent trace visibility
Observability	Lakewatch: suspicious activity / AI incident investigation	VendorNative	Delta Lake-based

Assessment: If an org runs OSS/Llama models on Databricks, Unity AI Gateway provides the richest programmatic control surface for MCP governance, agentic tool policy, and spend — **rivaling or exceeding hyperscaler offerings.**

PERPLEXITY ENTERPRISE PRO

Docs: <https://www.perplexity.ai/enterprise> | <https://www.perplexity.ai/hub/blog/how-perplexity-enterprise-pro-keeps-your-data-secure>

Standout: real-time webhook audit log — best-in-class observability for smaller vendors.

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	SSO/SAML 2.0 (Okta, Azure AD, Google Workspace)	VendorNative	Enterprise Pro

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	SCIM provisioning	VendorNative	Enterprise Pro
Data-Policy	No training on enterprise data	VendorNative (default)	Business and Enterprise
Data-Policy	MFA + short-lived session credentials	VendorNative	
Observability	Audit logs via real-time webhook (HTTP POST)	VendorNative	50+ seat requirement; captures: user queries, agentic steps/answers, file access, connector usage, admin settings changes; SIEM integrations (Splunk, Azure Sentinel, Datadog)
Observability	Agentic action / connector audit	VendorNative	Audit log includes agentic step visibility
Budget	Seat-based billing only	MonitorOnly	No per-use-case spend cap API

GROQ

Docs: <https://console.groq.com/docs/spend-limits> | <https://console.groq.com/docs/rate-limits>

Very limited enterprise surface. Spend controls are UI-only.

PLANE	FEATURE	STUB	NOTES
Budget	Monthly org-level spend cap	VendorNative (UI only)	Dashboard (Settings → Billing → Limits); no programmatic API
Budget	Alert thresholds at 50%/75%/90%	VendorNative (UI only)	Dashboard
Developer-Workflow	Org-level rate limits (shared)	VendorNative	All API keys in org share limits
Data-Policy	Training opt-out	Unverified	Not documented in public docs

IBM WATSONX

Docs: <https://www.ibm.com/products/watsonx-governance> | <https://www.ibm.com/docs/en/watsonx/saas>

Strongest governance story for regulated industries. Deep but complex.

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	Provider/model/secrets/ policy configuration (admin UI v2.4)	VendorNative	watsonx.ai

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	Agent catalog publishing + centralized credential management	VendorNative	watsonx Orchestrate
Developer-Workflow	Multi-tenancy with isolated tenants	VendorNative	watsonx.ai v2.4
Data-Policy	Hybrid multi-vendor GRC governance framework	VendorNative	watsonx.governance
Budget	Usage controls in admin UI	VendorNative	watsonx.ai v2.4
Observability	Agent Monitoring & Insights (decision/behavior/performance tracking, threshold alerts)	VendorNative	Q1 2026

XAI (GROK BUSINESS / ENTERPRISE)

Docs: <https://x.ai/api>

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	Team management, shared prompts/projects	VendorNative	Grok Business
Developer-Workflow	SSO/SAML, SCIM directory sync	VendorNative	Enterprise tier
Data-Policy	No training on business data (default)	VendorNative	Business and Enterprise
Budget	Custom rate limits + monthly invoice billing	VendorNative	Enterprise tier; negotiated
Observability	Audit logging	Unverified	Enterprise tier; programmatic API unconfirmed

3.12 ChatGPT Enterprise (distinct from the OpenAI API platform)

ChatGPT Enterprise is OpenAI's end-user product (not the API), with its own admin surface — materially stronger on identity and compliance observability than the API platform.

Docs: <https://openai.com/index/new-tools-for-chatgpt-enterprise/> · <https://help.openai.com/en/articles/9261474-openai-compliance-platform-for-enterprise-and-edu-customers>

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	SSO/SAML, SCIM, IP allowlists, custom roles, directory-synced groups	VendorNative	Managed via the Global Admin Console (<code>admin.openai.com</code>)

PLANE	FEATURE	STUB	NOTES
			spanning multiple workspaces
Developer-Workflow	App/action enable-disable; GPT/connector governance	VendorNative	Global Admin Console
Data-Policy	Data residency, CMEK, training opt-out (default)	VendorNative	
Budget	Seat-based; per-token caps	MonitorOnly	Seat license model; no per-token budgeting
Observability	Compliance Logs Platform	VendorNative (strong)	Immutable, time-windowed JSONL export: conversations, files, Admin Audit, User Authentication, Codex/agent usage; minutes-level latency

3.13 Gemini Enterprise / Gemini in Google Workspace (distinct from Vertex AI)

The Workspace-side Gemini admin surface, governed from the Google Workspace Admin console — separate from Vertex AI's Cloud IAM surface.

Docs: <https://workspace.google.com/blog/ai-and-machine-learning/enterprise-security-controls-google-workspace-gemini> · <https://docs.cloud.google.com/gemini/enterprise/docs/release-notes>

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	Service on/off + data-sharing toggles; AI control center (May 2026) governing AI/agent access to Workspace data	VendorNative	Workspace Admin console → Generative AI section
Data-Policy	DLP via Chrome Enterprise Premium (PII masking, copy/paste/upload control); Privacy Hub	VendorNative	
Observability	Gemini audit logs via Reports API (Drive-access events); security investigation tool; Vault eDiscovery for prompts/responses	VendorNative	
Budget	Seat/subscription	MonitorOnly	No per-interaction budgeting

3.14 Amazon Q (Business / Developer)

AWS’s end-user assistant family, governed through AWS IAM Identity Center — distinct from Bedrock.

Docs: <https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/security-iam.html>

PLANE	FEATURE	STUB	NOTES
Developer-Workflow	IAM Identity Center trusted identity propagation; admin API for app/connector/guardrail config	VendorNative	
Data-Policy	Admin-configurable guardrails ; automatic data residency; KMS CMEK	VendorNative	
Observability	CloudTrail audit trails with propagated user identity	VendorNative	
Budget	Subscription tiers	MonitorOnly	Per-token caps require proxy

4. Cross-Vendor Synthesis

4.1 Where Control Planes Converge

PLANE	CONVERGED CAPABILITY	VENDOR SUPPORT
Developer-Workflow	API key / service account management	All vendors; granularity varies
Developer-Workflow	Basic user/role management	Most enterprise vendors
Data-Policy	Training data opt-out	All enterprise vendors; most contractual, some API
Data-Policy	Content safety / guardrails	OpenAI, Azure, Bedrock, Google, GitHub Copilot
Budget	Usage/cost metering in dashboard	All vendors
Budget	Rate limiting (vendor-enforced)	All vendors; not all expose programmatic control
Observability	Usage metrics	All vendors; granularity varies

4.2 Where Control Planes Diverge (Critical Gaps)

GAP	AFFECTED VENDORS	AIR-TRAFFIC RESPONSE
No org-level model access API	Anthropic, Cohere, Mistral, Llama hosts	EnvManaged for agentic use (managed settings allow-list); ProxyEnforced for raw-API enforcement
No hard spend cap	AWS Bedrock, Anthropic, Cohere, Mistral, GitHub Copilot (credit model)	MonitorOnly + soft-alert by default; hard mid-request stop is ProxyEnforced (optional gateway)
No org-level content safety enforcement	Anthropic, OpenAI, Cohere, Mistral; Google Vertex unless Model Armor adopted	VendorNative where a guardrail API exists (Azure/Bedrock/Model Armor); else ProxyEnforced
No audit log API	Anthropic (API tier), Cohere, Mistral (panel-only, no export), Groq, xAI, Together AI	MonitorOnly : read platform audit surfaces where they exist; full request capture via the optional gateway
No MCP / agentic primitive controls	Most vendors. Exceptions: Databricks Unity AI Gateway, Bedrock AgentCore, GitHub MCP portal, Mistral per-workspace MCP Connectors	EnvManaged : pushed via the environment's managed config (Claude Code managed settings, branch protection, CI) — no proxy. Primary Air-Traffic value
No cross-vendor normalized audit stream	Industry-wide	Air-Traffic unifies into ops-observation-batch/v1
No cross-vendor spend aggregation	Industry-wide	Air-Traffic budget plane aggregates across vendor keys
No rigor-drift detection	Industry-wide	Air-Traffic computes policy-vs-actual diff
Request/response logs not customer-accessible	OpenAI (API), Anthropic (API), Cohere, Mistral, GitHub Copilot	Read platform audit surfaces where they exist (GitHub agentic audit, Claude Code telemetry, ChatGPT Enterprise Compliance Logs); full prompt/completion capture needs the optional gateway

4.3 Native API Depth by Vendor

VENDOR	DEVELOPER-WORKFLOW NATIVE	DATA-POLICY NATIVE	BUDGET NATIVE	OBSERVABILITY NATIVE	OVERALL
AWS Bedrock	★★★★	★★★★★	★★★★☆	★★★★★	Best-in-class infra; no spend cap
Azure OpenAI	★★★★	★★★★	★★★★	★★★★	Full enterprise integration

VENDOR	DEVELOPER-WORKFLOW NATIVE	DATA-POLICY NATIVE	BUDGET NATIVE	OBSERVABILITY NATIVE	OVERALL
Google Vertex	★★★★	★★★★	★★★☆☆	★★★★★	Deep; safety filter gap; spend caps in preview
OpenAI	★★★☆☆	★★☆☆☆	★★★☆☆	★★★☆☆	Good admin API; ZDR and content policy gaps
Anthropic	★★☆☆☆	★★★★☆	★★☆☆☆	★★★☆☆	Compliance API strong; workflow controls weak
GitHub Copilot	★★★★	★★★★☆	★★★☆☆	★★★★	Agent control plane + agentic audit GA 2026; portal-only policy toggles
M365 Copilot	★★★☆☆	★★★★★	★☆☆☆☆	★★★★	Purview deep; no budget controls
Mistral AI	★★★★	★★★★☆	★★★☆☆	★★☆☆☆	Best Admin API among small vendors; rare native per-workspace MCP control; audit export gap
Cohere	★☆☆☆☆	★★☆☆☆	★★☆☆☆	★☆☆☆☆	Mostly dashboard; strong on-prem option
Databricks	★★★★★	★★★★	★★★★★	★★★★★	Best for agentic/OSS if already on Databricks

4.4 Abstractions the Unified Layer Must Provide

1. **Policy-as-Code contract** — a single YAML policy that declares intended state across all three planes for all vendors; Air-Traffic computes the diff between intent and current state continuously.
2. **Normalized audit stream** — every request transiting Air-Traffic is logged in `ops-observation-batch/v1` with `plane`, `vendor`, `user`, `team`, `model`, and `control_surface` dimensions. This is the only complete, vendor-agnostic audit trail.
3. **Spend aggregation** — a virtual budget spanning multiple vendor keys; hard/soft cap enforcement fires before the first vendor call that would exceed the cap, regardless of vendor.

- 4. **Per-capability disposition signal** — every control carries its mechanism (`VendorNative` / `EnvManaged` / `ProxyEnforced` / `MonitorOnly`) as a structured signal (not an error), so the scorecard tile renders the how distinctly — and, for `EnvManaged` , an **enforcement-confidence** level (server-side-enforced / MDM-locked / seed-and-drift-only).
- 5. **MCP and agentic primitive registry** — a managed catalog of approved MCP servers, hooks, and scheduled jobs **distributed as managed config** into every dev/agent environment (Claude Code managed settings, branch protection, CI) and read back for drift, independent of vendor — `EnvManaged` , no request interception.

4.5 What Changed in 2026 (gaps that narrowed — and the ones that didn't)

The vendor landscape moved fast in 2026. Several gaps this analysis would have flagged a year ago have partially closed — and tracking that movement is itself a reason a control plane must continuously re-verify capabilities rather than hard-code them:

GAP	2026 MOVEMENT	NET
Agentic/MCP observability	GitHub agent control plane + agentic audit events (GA Feb 2026); OpenAI Codex/agent activity in Compliance Logs; Mistral per-workspace MCP Connectors	Narrowed — but still no cross-vendor agentic audit
Org-enforced content safety	Google Model Armor GA; Bedrock cross-account Guardrails; AgentCore Guardrails-in-policy	Narrowed for the hyperscalers; still open for OpenAI/Anthropic/Cohere
Data-policy self-serve	Together AI self-serve ZDR + opt-in training; OpenAI EU residency-with-ZDR	Narrowed
Hard cross-vendor spend cap	None — AWS itself prescribes a proxy/Lambda/DynamoDB budget check	Confirmed wide open
Cross-vendor normalized policy + audit	None — every vendor remains an island	Confirmed wide open
Rigor-drift detection	None offered by any vendor or gateway	Confirmed wide open

The durable white space is precisely the cross-vendor, dual-control (native + proxy), declare-and-prove layer — which no single vendor has any incentive to build.

4.6 Prior Art: the AI-Gateway Landscape and Air-Traffic’s White Space

A mature field of “AI gateways” / “LLM gateways” already exists. Understanding it is essential — both because it validates Air-Traffic’s proxy data-plane design and because it sharpens what is genuinely differentiated.

PRODUCT	DEPLOYMENT	BUDGET ENFORCEMENT	OBSERVABILITY	POLICY / GUARDRAILS	MCP / AGENTIC
LiteLLM		Hard caps via Redis cross-pod counter;	Postgres spend logs; OTel/	Guardrail hooks (PII, prompt-	Routes agent traffic; no

PRODUCT	DEPLOYMENT	BUDGET ENFORCEMENT	OBSERVABILITY	POLICY / GUARDRAILS	MCP / AGENTIC
	OSS self-host proxy + enterprise	DB fail-closed; per key/user/team/org/tag; multi-window	Langfuse/Helicone export	injection, Bedrock passthrough)	MCP-native governance
Portkey	OSS gateway (MIT) + SaaS	Budget caps + rate limits per virtual key/workspace	OTel-compliant tracing; logs model + MCP tool calls	50+ guardrails: PII, content filter, LLM-as-judge	MCP Gateway: central registry; tool calls governed like model calls
Cloudflare AI Gateway	SaaS (edge)	Rate limiting by model/provider/metadata; no hard \$ cap	Per-request logs (toggle body capture)	Dynamic routing; newer guardrails	None notable
Kong AI Gateway	Self-host / hybrid	Token + request rate limiting; cost-based routing	Kong analytics / OTel	AI PII Sanitizer: 20 categories, 9 languages, req + resp	MCP security integration (3.10+)
TrueFoundry	Self-host (K8s) / SaaS	Budgets/quotas per team/user/model/app/env; throttle / downgrade-model / block	Usage + perf monitoring	RBAC, OAuth2, virtual keys, residency rules	Agent + MCP Gateway
Databricks Mosaic/Unity AI Gateway	Databricks-native	Permission + QPM limits; system-table usage	Payload logging via inference tables	Predefined + custom guardrails (PII, toxicity)	Unity AI Gateway extends to MCP/agent governance
Helicone	OSS proxy + SaaS	Custom rate limits; cost analytics	Full req/resp logging at <1ms self-host overhead	Caching, rate limiting	Minimal
Langfuse	OSS self-host	None (not a gateway)	Best-in-class tracing; native OTLP endpoint	Evals (LLM-as-judge)	Traces agent/tool steps
OpenRouter	SaaS	One credit balance; no enterprise budget governance	Basic dashboard	Minimal	None

The white space (validated against the field): every incumbent is a runtime data-plane proxy. They govern the request as it passes through — virtual keys + Redis counters + guardrail hooks. **None of them reach back into the vendors' native admin APIs** to set training opt-out, ZDR, native content filters, native spend alerts, or audit-log export — the **VendorNative** half of Air-Traffic's model. Air-

Traffic is the only design that unifies **both control surfaces** (drive the vendor’s native admin API where one exists; enforce at the proxy where one doesn’t, with an explicit per-capability disposition), and layers on three things no gateway ships:

- 1. **Cross-vendor policy-as-code** — one declared intent reconciled against actual state across both the proxy and every vendor’s admin plane.
- 2. **One-click industry rigor baselines** that auto-split into native-where-possible / proxy-where-not.
- 3. **Rigor-drift detection** — flags when a console change, a vendor default change, or a bypassed proxy diverges from declared policy.

The gateways answer “control how traffic flows right now.” Air-Traffic answers “declare and continuously prove the governance posture of the whole AI estate, across both the proxy and the vendors’ own admin planes.” Closest competitors: **Portkey** (best guardrails + MCP + OTel, but proxy-only, no native-admin control, no drift/baseline) and **Databricks Unity AI Gateway** (richest governance, but only inside Databricks).

Design implication: Air-Traffic’s proxy data-plane should adopt the field’s proven primitives rather than reinvent them — OpenAI-compatible passthrough, virtual keys, Redis cross-pod counters with DB fail-closed, ordered guardrail hooks, threshold actions (throttle/downgrade/block), an MCP server registry, and the **OpenTelemetry GenAI semantic conventions** as the observability schema (so the normalized stream is SIEM/OTel-ingestible and deep tracing can be offloaded to Langfuse via OTLP). These are specified in `air-traffic-system-design.md`.

5. Air-Traffic API: VendorAdapter Architecture

5.1 Design Principles

Air-Traffic’s API is modeled directly on the `it-scorecard` connector/emitter pattern:

IT-SCORECARD CONCEPT	AIR-TRAFFIC ANALOG
<code>Connector</code>	<code>VendorAdapter</code> (one per vendor/platform)
<code>Mode: synthetic</code>	Walk-based stub signal (same as <code>emitSynthetic</code>)
<code>Mode: proxy</code>	Real work: control half drives the vendor admin API; env half pushes managed config
<code>Mode: disabled</code>	Adapter registered but silent
<code>Emit()</code>	Produces <code>ops-observation-batch/v1</code> batches per adapter
<code>/api/observations</code>	Unchanged — Air-Traffic scorecard consumes same endpoint
<code>catalog</code>	<code>AdapterManifest</code> — capability entries with dispositions
<code>proxy_not_normalized</code> error	<code>env_managed</code> / <code>vendor_native</code> honesty signal

Each adapter has a **control half** (drives the vendor admin API for `vendor_native` controls) and an **env half** (`RenderManagedConfig` + `ReadEnvState` for `env_managed` controls). Request forwarding (`Forward`) is **not** in the core interface — it belongs to the optional inference gateway (system-design §11), keeping the spine off the request path.

5.2 VendorAdapter Interface

```

// Package adapter defines the contract every AI-vendor connector must implement.
// Air-Traffic mirrors the it-scorecard connector/emitter pattern:
//   - synthetic mode → walk-based stub signal (like it-scorecard's emitSynthetic)
//   - proxy mode     → drive vendor admin API (VendorNative) + push env config (EnvManaged)
//   - disabled       → registered but silent
package adapter

import (
    "context"
    "time"

    "it-scorecard/internal/model" // reuse ops-observation-batch/v1 model directly
)

// SignalKind declares how a control or observation is fulfilled.
type SignalKind string

const (
    // SignalVendorNative: vendor exposes a programmatic admin API; the control half drives it.
    SignalVendorNative SignalKind = "vendor_native"
    // SignalEnvManaged: no vendor API; the env half pushes managed config into the dev/agent
    // environment (MCP, hooks, code-review, cron) and reads it back. No request interception.
    SignalEnvManaged SignalKind = "env_managed"
    // SignalProxyEnforced: runtime-only control requiring the OPTIONAL inline gateway
    // (hard cross-vendor caps, per-request PII scrub). Off by default; not on the spine.
    SignalProxyEnforced SignalKind = "proxy_enforced"
    // SignalUnverified: capability status could not be confirmed from vendor docs.
    SignalUnverified SignalKind = "unverified"
    // SignalUnsupported: vendor explicitly does not support this control.
    SignalUnsupported SignalKind = "unsupported"
)

// Signal mirrors the it-scorecard error entry shape (scope/code/message/retryable)
// but adds Kind so callers know the mechanism (vendor API, env-config push, or optional gateway).
type Signal struct {
    Kind      SignalKind `json:"kind"`
    Code      string    `json:"code,omitempty"`
    Message   string    `json:"message,omitempty"`
    Retryable bool      `json:"retryable"`
}

type Plane string

const (
    PlaneDeveloperWorkflow Plane = "developer_workflow"
    PlaneDataPolicy         Plane = "data_policy"
    PlaneBudget             Plane = "budget"
)

// CapabilityEntry declares one control or observable and its stub disposition.
type CapabilityEntry struct {
    Name      string
    Plane     Plane
    SignalKind SignalKind
    Notes     string
}

// AdapterManifest is the machine-readable version of the vendor analysis tables.
type AdapterManifest struct {
    VendorID      string
    DisplayName   string

```

```

APIVersion    string
Mode          model.Mode
Capabilities []CapabilityEntry
}

// VendorAdapter is the unified contract every AI-vendor connector must satisfy.
// It covers all three control planes + observability, and emits
// it-scorecard-compatible ops-observation-batch/v1 batches.
type VendorAdapter interface {

    // --- Identity & manifest ---
    VendorID()    string
    APIVersion()  string
    Manifest()    AdapterManifest

    // --- Developer-Workflow Plane ---
    ListModels(ctx context.Context, req ListModelsRequest) (ListModelsResponse, Signal, error)
    SetModelAccess(ctx context.Context, req ModelAccessRequest) (Signal, error)
    GetAgentPolicies(ctx context.Context) (AgentPolicies, Signal, error)
    SetAgentPolicies(ctx context.Context, p AgentPolicies) (Signal, error)

    // --- Data-Policy Plane ---
    GetDataPolicy(ctx context.Context) (DataPolicy, Signal, error)
    SetDataPolicy(ctx context.Context, p DataPolicy) (Signal, error)
    GetRetentionPolicy(ctx context.Context) (RetentionPolicy, Signal, error)
    SetRetentionPolicy(ctx context.Context, p RetentionPolicy) (Signal, error)
    GetContentSafetyPolicy(ctx context.Context) (ContentSafetyPolicy, Signal, error)
    SetContentSafetyPolicy(ctx context.Context, p ContentSafetyPolicy) (Signal, error)

    // --- Budget Plane ---
    GetSpendLimits(ctx context.Context) (SpendLimits, Signal, error)
    SetSpendLimits(ctx context.Context, limits SpendLimits) (Signal, error)
    GetUsage(ctx context.Context, req UsageRequest) (UsageReport, Signal, error)

    // --- Env half (env_managed controls: MCP, hooks, code-review, cron) ---
    // RenderManagedConfig turns the policy into a platform-specific managed-config
    // artifact (e.g. Claude Code managed-settings.json, a GitHub branch-protection
    // rule). The config distributor pushes it; nothing is intercepted at runtime.
    RenderManagedConfig(ctx context.Context, p AgentPolicies) (ManagedArtifact, Signal, error)
    // ReadEnvState reads the effective state back for drift comparison.
    ReadEnvState(ctx context.Context) (EnvState, Signal, error)

    // --- Observability ---
    GetAuditLogs(ctx context.Context, req AuditLogRequest) (AuditLogBatch, Signal, error)
    GetUsageMetrics(ctx context.Context, req MetricsRequest) (MetricsBatch, Signal, error)

    // --- it-scorecard compatible emission ---
    // Emit generates one ops-observation-batch/v1 batch representing the current
    // state of all three control planes for this vendor.
    // - synthetic mode → walk-based metric values (like emitSynthetic)
    // - proxy mode      → drives vendor APIs + reads env-config state; normalizes
    // - env_managed controls → kind=state, fixture="env_managed"
    Emit(ctx context.Context, ts time.Time) (model.ObservationRecord, error)
}

// Note: there is no Forward() here. Request forwarding is the optional inference
// gateway's job (system-design §11), not the core adapter's – the spine never
// sits on the inference critical path.

```

5.3 Signal Propagation Pattern

Mirroring it-scorecard's `emitProxy` honesty:

```
proxy mode + vendor_native control → real admin API call → normalize → observation (complete: true)

proxy mode + env_managed control → render + distribute managed config → read back → signal:
{
  "scope":          "control",
  "code":           "env_managed",
  "message":        "MCP allow/deny pushed via Claude Code managed settings;
                    effective state read back for drift",
  "retryable":      false,
  "http_status":    null,
  "retry_after_seconds": null
}

proxy mode + proxy_enforced control → only if the optional gateway is on; else emit
                                   "monitor_only" (observe + alert, no hard stop)

synthetic mode                    → walk-based metric values → observation (complete: true)
```

The scorecard tile for “Anthropic MCP policy” renders teal with label `EnvManaged` — it’s working via config distribution, not vendor-delegated and not proxied. A `proxy_enforced` control with the gateway off renders amber `monitor_only`, never a fabricated green. Honest engineering, not error state.

5.4 Example Manifest: OpenAI Adapter

```
func (a *OpenAIAdapter) Manifest() AdapterManifest {
    return AdapterManifest{
        VendorID:      "openai",
        DisplayName:    "OpenAI",
        APIVersion:     "2024-02-01",
        Mode:          a.mode,
        Capabilities: []CapabilityEntry{
            // Developer-Workflow
            {Name: "model_access", Plane: PlaneDeveloperWorkflow,
             SignalKind: SignalVendorNative,
             Notes: "PATCH /admin/organization/projects/{id}/model_permissions"},
            {Name: "mcp_server_policy", Plane: PlaneDeveloperWorkflow,
             SignalKind: SignalEnvManaged,
             Notes: "pushed via agent-environment managed config; read back for drift"},
            {Name: "agentic_hooks", Plane: PlaneDeveloperWorkflow,
             SignalKind: SignalEnvManaged},

            // Data-Policy
            {Name: "training_opt_out", Plane: PlaneDataPolicy,
             SignalKind: SignalVendorNative, Notes: "contractual default"},
            {Name: "zero_data_retention", Plane: PlaneDataPolicy,
             SignalKind: SignalVendorNative,
             Notes: "EU residency is in-region with ZDR; else enterprise-negotiated"},
            {Name: "content_safety_org_level", Plane: PlaneDataPolicy,
             SignalKind: SignalProxyEnforced, Notes: "optional gateway; else monitor_only"},
            {Name: "pii_redaction", Plane: PlaneDataPolicy,
             SignalKind: SignalProxyEnforced, Notes: "optional gateway (request-path)"},

            // Budget
            {Name: "spend_alerts", Plane: PlaneBudget,
             SignalKind: SignalVendorNative,
             Notes: "POST /admin/organization/projects/{id}/spend_alerts"},
            {Name: "hard_spend_cap", Plane: PlaneBudget,
             SignalKind: SignalProxyEnforced, Notes: "no native hard cap; optional gateway, else monitor_only"},
            {Name: "usage_metering", Plane: PlaneBudget,
             SignalKind: SignalVendorNative, Notes: "GET /v1/organization/usage"},
        },
    }
}
```

6. Baseline Configuration Profiles

6.1 Profile Summary Matrix

PROFILE	MODEL ACCESS	TRAINING OPT-OUT	ZDR	PII REDACT	CONTENT SAFETY	RETENTION	ORG CAP	USER CAP	CORE RE
General SaaS	Open	ON	OFF	OFF	Standard	30d	\$10K	\$200	No ag re
Fintech		ON		ON	Elevated	7d	\$50K	\$500	Hu re

PROFILE	MODEL ACCESS	TRAINING OPT-OUT	ZDR	PII REDACT	CONTENT SAFETY	RETENTION	ORG CAP	USER CAP	CO RE
	Restricted (prod only)		ON (where native)						(fi flo
Healthcare	BAA-only allowlist	ON	ON (enforced)	ON + PHI	Maximum	Zero	Board-approved	Human-required ALL	
Gov/Infra	FedRAMP/ on-prem only	ON	ON (enforced)	ON	Maximum	Audit-controlled	Agency-approved	Human-required ALL	

6.2 General SaaS — Standard

DEVELOPER-WORKFLOW

SETTING	VALUE
Model access	All current-generation models available to all team members
MCP servers	Approved list: developer tooling servers only (filesystem, git, browser, code execution)
Code review level	None for internal tools; agentic-reviewer for customer-facing commits
Test requirements	None mandatory (team discretion)
Cron jobs	Allowed; manager approval for prod-scope jobs

DATA-POLICY

SETTING	STUB
Training opt-out: ON (all vendors)	VendorNative where available; MonitorOnly otherwise
Data retention: 30-day	Vendor default
ZDR: OFF	—
Request/response logging: ON (debugging)	Vendor or ProxyEnforced
PII redaction: OFF	—
Content safety: Standard (Block at high severity)	VendorNative where available

BUDGET

SETTING	VALUE
Org monthly cap	\$10,000 hard cap

SETTING	VALUE
Team cap	\$1,000/team/month soft cap
Per-user cap	\$200/user/month soft cap
Rate limit	100K tokens/min per user
Alert threshold	80% of any cap

6.3 Financial Services — Elevated🔒🔒

DEVELOPER-WORKFLOW

SETTING	VALUE
Model access	Production-tier models only; no experimental/preview; approval required for new models
MCP servers	Strict allowlist: internal code/DB read-only only; no external API servers
Code review level	Human-required for financial calculations, customer-data access, payment flows
Test requirements	≥90% coverage on agent-authored code before merge
Cron jobs	Require security review + change management ticket

DATA-POLICY

SETTING	STUB
Training opt-out: ON (mandatory)	VendorNative / MonitorOnly
Data retention: 7-day max	MonitorOnly (verify vendor setting + gate); ProxyEnforced only if capturing logs
ZDR: ON where available	VendorNative (OpenAI Enterprise, Azure) / MonitorOnly
Request/response logging: ON (encrypted, 90-day audit archive)	Vendor logs + Air-Traffic gateway log (optional module)
PII redaction: ON (account numbers, SSNs, card numbers)	ProxyEnforced
Content safety: Elevated (Block at medium severity)	VendorNative (Azure, Bedrock) / ProxyEnforced
Data residency: US or EU only	VendorNative (Azure, Bedrock, Vertex) / MonitorOnly

BUDGET

SETTING	VALUE
Org monthly cap	\$50,000 hard cap (CFO approval to change)
Team cap	\$5,000/team/month hard cap
Per-user cap	\$500/user/month hard cap
Rate limit	50K tokens/min per user
Alert threshold	60% soft; 90% hard + freeze
Cost attribution	Mandatory: cost-center + regulatory scope per request

6.4 Healthcare — Strict

DEVELOPER-WORKFLOW

SETTING	VALUE
Model access	BAA-signed vendors only (Azure OpenAI, AWS Bedrock, Google Vertex in covered regions)
MCP servers	Blocked by default; whitelist-only; no external network access
Code review level	Human-required for ALL agent-authored code; agentic-reviewer supplemental only
Test requirements	100% coverage on patient-data paths; compliance scan required
Cron jobs	Prohibited unless approved by CISO + compliance officer

DATA-POLICY

SETTING	STUB
Training opt-out: ON (mandatory; BAA-signed vendors only)	VendorNative / MonitorOnly
Data retention: ZERO	VendorNative (ZDR) / ProxyEnforced (proxy blocks logging)
ZDR: ON (enforced at proxy if not native)	VendorNative + MonitorOnly
Request/response logging: Air-Traffic gateway log only, PHI scrubbed before logging	ProxyEnforced (gateway required for this tier)
PII/PHI redaction: ON (names, DOBs, MRNs, SSNs, diagnoses, medications)	ProxyEnforced
Content safety: Maximum	VendorNative / ProxyEnforced

SETTING	STUB
Data residency: US only, HIPAA-covered regions	VendorNative (Azure, Bedrock, Vertex) / MonitorOnly
BAA requirement: Only BAA-signed vendors enabled	Air-Traffic manifest flag

BUDGET

SETTING	VALUE
Org monthly cap	Board-approved hard cap; no override path
Per-user cap	\$100/user/month hard cap
Rate limit	20K tokens/min per user
Alert threshold	50% alert; 80% hard throttle
Cost attribution	Department + application + data-sensitivity level per request

7. Observability Layer Design

7.1 The Normalized Signal

Air-Traffic extends `ops-observation-batch/v1` with AI-specific dimensions. Every `VendorAdapter` observation carries:

```
{
  "contract": "ops-observation-batch/v1",
  "batch_id": "uuid",
  "connector": {
    "type": "ai-vendor",
    "instance": "openai",
    "api_version": "2024-02-01"
  },
  "observations": [
    {
      "kind": "metric",
      "signal": {
        "name": "tokens_in",
        "value": 142500,
        "unit": "tokens",
        "status": "green",
        "severity": "info"
      },
      "dimensions": {
        "plane": "budget",
        "vendor": "openai",
        "team": "platform-engineering",
        "model": "gpt-4o",
        "control_surface": "VendorNative"
      },
      "provenance": {
        "fixture": "vendor_native",
        "source_url": "https://api.openai.com/v1/organization/usage"
      }
    }
  ]
}
```

7.2 Observability Coverage by Plane

BUDGET PLANE (BEST NATIVE COVERAGE)

SIGNAL	DIM	OPENAI	ANTHROPIC	AZURE	BEDROCK	GOOGLE	GITHUB COPILOT
Tokens in/out	vendor, model, team	✓	✓	✓	✓	✓	seat-based
Cost (USD)	vendor, team	✓	✓	✓	✓	✓	✓
Cap utilization %	—	Air-Traffic	Air-Traffic	Air-Traffic	Air-Traffic	Air-Traffic	Air-Traffic

DATA-POLICY PLANE

SIGNAL	COVERAGE
Content safety triggers	VendorNative (Azure, Bedrock, Model Armor)
ZDR compliance rate	Air-Traffic computed

SIGNAL	COVERAGE
Policy drift events	Air-Traffic computed
PII redaction events	ProxyEnforced (optional gateway only)

DEVELOPER-WORKFLOW PLANE (OBSERVED FROM PLATFORM AUDIT SURFACES, NOT A PROXY)

SIGNAL	COVERAGE
MCP allow/deny state	EnvManaged (read back from managed config)
Agentic action / tool-call logs	VendorNative platform logs (GitHub 2026 agentic audit, Claude Code telemetry, ChatGPT Enterprise Compliance Logs)
Code-review gate outcomes	VendorNative (GitHub branch-protection + checks API)
Model access denials	EnvManaged (managed config) / ProxyEnforced (if gateway on)

7.3 Unified Audit Stream Schema

```
AuditEvent {
  id, timestamp, actor, action, resource, plane, vendor,
  control_surface: "VendorNative" | "EnvManaged" | "ProxyEnforced",
  before, after,      // state transition for control changes
  request_id          // correlation with usage events
}
```

Events: policy changes, model access grants/revocations, spend cap modifications, MCP server allow/deny changes, user role changes, credential updates, all Air-Traffic-mediated actions (env-config pushes + optional-gateway events), and normalized events from all vendor native audit APIs (OpenAI Audit Logs, Azure Activity Log, GitHub Audit Log, CloudTrail, M365 Purview Audit).

8. UI Layout Proposals

8.1 Rigor Console — Simple Configuration View

Primary user: CISO, IT Administrator, Data Privacy Officer
Goal: Set a baseline rigor level for each control surface in one glance.

AIR-TRAFFICControl Plane

[Profile: Fintech]

VENDORS

OpenAI

Anthropic

Azure

Bedrock

Gemini

ALL

DATA POLICY	RIGOR	COVERAGE
Data Retention		6/10 native
Training Opt-Out		9/10 native
PII Redaction		+ proxy (opt)
Content Safety		5/10 native
Data Residency		4/10 native
DEVELOPER WORKFLOW		
Model Access		3/10 native
MCP Server Policy		env-managed
Code Review Level		env-managed
Agent Permissions		env-managed
BUDGET		
Org Spend Cap		7/10 native
Per-User Spend Cap		+ proxy (opt)
Rate Limits		6/10 native
Budget Alerting		5/10 native

Native

EnvManaged

Proxy(opt)

[Apply Profile]

[Save]

Each slider maps to / / . Coverage marks: **Native** (count of vendors with a native admin API), **EnvManaged** (pushed as managed config into the dev/agent environment — the agentic primitives), **Proxy(opt)** (needs the optional inference gateway). A row prompts a confirmation that it requires enabling the gateway module; everything else applies with no request interception.

8.2 Policy Editor — Advanced Controls View

Primary user: Platform Engineer, Security Engineer
Goal: Expand each rigor level into individual controls for fine-tuning; per-vendor overrides.



8.3 Cost & Usage Explorer

Primary user: FinOps, Engineering Leadership, Team Leads

Goal: Spend velocity, per-team attribution, cap utilization, cost anomaly alerts across all vendors.



8.4 Flight Deck — Vendor Status Board

Primary user: Platform Engineer, Security Engineer

Goal: One-glance status of every connected vendor: connectivity, policy sync state, observability freshness, drift from declared baseline.

AIR-TRAFFIC Vendor Status Board				06-29 14:32 UTC	
VENDOR	STATUS	POLICY	OBS FRESH	DRIFT	
OpenAI	●green	✓ synced	14s ago	none	
Anthropic	●green	✓ synced	16s ago	none	
Azure OAI	●green	✓ synced	12s ago	none	
AWS Bedrock	●green	✓ synced	11s ago	none	
Google Vertex	●green	✓ synced	18s ago	none	
GitHub Copilot	●amber	▲ 2 gaps	42s ago	seat policy drift	
M365 Copilot	●green	✓ synced	21s ago	none	
Cohere	●green	◆ env-mgd	19s ago	none (env)	
Mistral AI	●amber	▲ 1 gap	38s ago	ZDR unconfirmed	
Together AI	●red	✕ error	8m ago	connection failed	
● Native ◆ EnvManaged + Proxy(opt) ✕ Conn Error					
[+ Add Vendor] [Run Policy Sync] [Export Audit] [Manage Credentials]					

The DRIFT column is rigor-drift detection in action: it shows when a vendor’s config has diverged from the declared baseline, surfacing which controls are out of sync.

9. Killer Differentiating Features

Each feature below was checked against the existing AI-gateway field (§4.6). All five sit in confirmed white space: no incumbent gateway drives vendors' native admin APIs, and none ships cross-vendor policy-as-code, industry baselines, or drift detection.

9.1 Cross-Vendor Policy-as-Code

A single YAML/JSON document declares intended state across all three planes for all vendors. Air-Traffic continuously computes the diff between declared intent and actual state.

Why it wins: No competitor offers this. An admin declares `mcp.allow: [...]` and `training_opt_out: true` once; Air-Traffic sets training opt-out via each vendor's native API (or confirms the contractual default), and pushes the MCP allow-list into every environment as managed config — with a coverage report showing exactly which controls are `VendorNative`, which are `EnvManaged`, and which would need the optional gateway. Policy reviews become a diff, not a 10-console survey.

```
# air-traffic-policy.yaml
vendor_defaults:
  training_opt_out: true
  content_safety: elevated
  data_retention_days: 7

vendors:
  openai:
    model_access:
      mode: allow_list
      allowed_models: [gpt-4o, gpt-4o-mini, o1, o3-mini]
    spend_alerts:
      threshold_cents: 100000
  anthropic:
    model_access:
      mode: allow_list          # EnvManaged via Claude Code managed settings; raw-API → optional gateway
      allowed_models: [claude-sonnet-4-6, claude-haiku-4-5]
  bedrock:
    content_safety:
      guardrail_id: g-12345abc  # VendorNative (cross-account Guardrails)
      apply_cross_account: true

agentics:                      # EnvManaged — pushed into dev/agent environments
  claude_code:
    mcp: { allow: [filesystem, git, internal-db-readonly], deny: ["*"] }
    hooks: { pre_commit: [run-tests, secret-scan] }
  github:
    code_review: { require_human_review: true, required_status_checks: [tests, coverage-90] }

budget:
  org_monthly_cap_usd:
    50000    # VendorNative soft caps + cross-vendor MONITOR; hard stop only if gateway on
  team_caps:
    platform-engineering: 10000
    product: 8000
```

9.2 One-Click Industry Baseline

A curated library of baseline configuration profiles that can be applied to the entire vendor estate in one click. Profiles cover Fintech 🏦 , Healthcare 🏥 , General SaaS 🏢 , Government 🏛️ .

Why it wins: Enterprise security teams spend weeks calibrating AI vendor settings. A baseline collapses that to 5 minutes. The profile applies what's natively possible (`VendorNative`), pushes the rest as managed environment config (`EnvManaged`), and flags the few controls that would need the optional gateway — the admin never needs to understand which vendor supports what. Baselines are version-controlled YAML in the policy-as-code repo.

9.3 Unified Spend Governance with Cross-Vendor Hard Caps

A virtual budget layer aggregates spend across all vendors in real time and enforces hard caps spanning vendor boundaries — stopping a user who has hit their \$500/month cap regardless of whether their next call is to OpenAI, Anthropic, or Bedrock.

Why it wins: Today, a user who hits their OpenAI limit just switches to Claude. Air-Traffic's budget plane is vendor-agnostic: the cap is on the user, not the key. FinOps gets a single P&L view for AI spend with per-team chargeback attribution that works across vendors. This is particularly critical given AWS Bedrock's complete lack of a native hard spend cap — AWS itself prescribes a proxy.

Scope note: Monitoring cross-vendor spend, setting native caps, and soft alerts are all spine features (no proxy). Only the **hard mid-request stop** requires the optional inference gateway (`ProxyEnforced` , system-design §11). With the gateway off, this feature degrades honestly to observe-and-alert — it never silently fails to stop a runaway.

9.4 Single Normalized Audit-Log Stream

A unified `ops-observation-batch/v1` event stream normalizing audit events from every vendor's native audit API (OpenAI Audit Logs — 51 event types; Azure Activity Log; GitHub Audit Log; CloudTrail; M365 Purview Unified Audit) plus Air-Traffic's own proxy events — searchable in one place with consistent field names.

Why it wins: Security teams today cannot answer “who called which AI model with what data, across our entire AI estate, between dates X and Y?” Air-Traffic answers from one API endpoint. The stream is SIEM-compatible (Splunk, Elastic, Datadog) via the `ops-observation-batch/v1` contract. Agentic action logs (MCP calls, hook executions, code review outcomes) are included — gaps that no vendor audit log covers today.

9.5 Rigor-Drift Detection

Air-Traffic continuously compares the declared policy against observed current state of each vendor's configuration. When a vendor setting drifts — because someone changed it in the console, a vendor updated their default behavior, or a new vendor was added without policy coverage — Air-Traffic surfaces a drift event in the audit stream and Flight Deck UI.

Why it wins: Compliance teams struggle to prove AI governance controls are continuously in effect. Rigor-drift detection makes governance a continuous process: every compliance audit starts with a 30-day Air-Traffic drift report showing policy-vs-actual history. The per-capability disposition means drift is detected regardless of mechanism — a console side-change to a `VendorNative` control, a developer overriding a seed-only `EnvManaged` setting, or an MDM-locked managed file going missing all degrade the tile to amber and fire a drift alert.

10. Appendix: Stub Disposition Reference

Coverage Statistics (across 10 major vendors surveyed)

PLANE	VENDORNATIVE	ENVMANAGED	PROXYENFORCED	MONITORONLY	UNVERIFIED
Developer-Workflow	~30%	~52%	~8%	~2%	~8%
Data-Policy	~48%	~2%	~22%	~14%	~14%
Budget	~50%	~0%	~14%	~26%	~10%
Observability	~55%	~5%	~8%	~15%	~17%

Mechanism mix by plane: the developer-workflow plane is dominated by `EnvManaged` (managed-config push), data-policy splits across `VendorNative` / `ProxyEnforced` / `MonitorOnly` , and budget is mostly `VendorNative` soft-controls + `MonitorOnly` with `ProxyEnforced` reserved for hard caps. Only ~8–22% of any plane needs the optional gateway.

The data proves the thesis: The majority of enterprise AI governance controls — particularly the developer-workflow plane (agentic primitives) — are still not natively controllable via vendor APIs. 2026 narrowed the agentic-observability and content-safety gaps for the hyperscalers (GitHub agent control plane, Model Armor, Bedrock cross-account Guardrails), but the **cross-vendor** gaps — hard spend caps that span vendors, one normalized policy/audit surface, and drift detection — are confirmed wide open. Air-Traffic closes them with two spine mechanisms — `VendorNative` (drive each vendor’s admin API) and `EnvManaged` (push managed config into the dev/agent environment) — neither of which is on the inference path. The optional `ProxyEnforced` gateway covers only the two residual runtime controls. The `VendorNative` half in particular is the part no existing AI gateway even attempts.

Confirmed Admin API Endpoints by Vendor

VENDOR	KEY PROGRAMMATIC SURFACES
OpenAI	<code>/admin/organization/projects/{id}/model_permissions ; /admin/organization/audit_logs ; /admin/organization/projects/{id}/spend_alerts ; /v1/organization/usage ; /v1/organization/costs</code>
Anthropic	

VENDOR	KEY PROGRAMMATIC SURFACES
	<code>/v1/organizations/ workspaces ; /v1/organizations/usage_report/ messages ; /v1/organizations/cost_report ; /v1/ compliance/activities ; /v1/compliance/chats</code>
Google Vertex	Cloud IAM API; Cloud Billing Budgets API; Cloud Quotas API (<code>serviceusage.quotas.update</code>); Cloud Audit Logs API; Vertex AI Model Monitoring API
AWS Bedrock	<code>PutModelInvocationLoggingConfiguration ; CreateGuardrail ; Organizations:BEDROCK_POLICY ;</code> CloudTrail data events; Service Quotas API; Cost Explorer API
Azure OpenAI	ARM deployment API; <code>GET .../usages</code> (Usages API); <code>GET .../modelCapacities</code> ; Diagnostic Settings (RequestResponseLog); Azure Monitor Metrics API
GitHub Copilot	<code>GET/PUT /enterprises/{enterprise}/copilot/ content_exclusion ; GET /enterprises/{enterprise}/copilot/metrics ; GET /enterprises/{enterprise}/copilot/billing/ seats ;</code> GitHub Audit Log API incl. agentic events (<code>actor_is_agent</code> , <code>agent_session.task</code> , <code>actor:Copilot</code>)
ChatGPT Enterprise	Global Admin Console (<code>admin.openai.com</code>); SCIM; Compliance Logs Platform (JSONL export: conversations, files, admin audit, auth, Codex usage)
Gemini Enterprise / Workspace	Workspace Admin console (Generative AI section); AI control center ; Gemini audit logs via Reports API ; Vault eDiscovery
Amazon Q	IAM Identity Center; admin API for app/connector/guardrail config; CloudTrail with propagated user identity
M365 Copilot	Graph Reports API (<code>/reports/microsoft365CopilotUsageUserDetail</code>); Purview Audit (Office 365 Mgmt Activity API); Graph license assignment API
Mistral AI	<code>/beta/admin/workspaces ; /beta/admin/users ; / beta/admin/api-keys ; /beta/admin/billing ; /beta/ admin/analytics</code> (all Preview, Enterprise-only)
Cohere	Dashboard-only for most; no confirmed programmatic admin API beyond key management
Together AI	Project-scoped key management; self-serve ZDR + training opt-in toggles (Privacy & Security); no programmatic billing/spend API confirmed

Document generated: June 2026. Verify vendor capabilities against live documentation before implementation — AI vendor admin APIs evolve frequently. All API docs URLs verified against live documentation during research.